DOSSIER DE PROJET

Application de gestion de commandes pour restaurant

Projet	Système de commandes restaurant
Architecture	Serveur Node.js central + clients React
Interfaces	Client / Caisse, Cuisinier, Serveur, Suivi client
Technologies	React, Node.js, Express, Socket.io, MongoDB

Document préparé pour une soutenance de type TP DWWM. Il décrit le besoin métier, l'architecture, les fonctionnalités et les compétences mobilisées.

1. Contexte et objectif

Le projet vise à centraliser la gestion des commandes d'un restaurant dans un système unique accessible par plusieurs rôles.

L'objectif principal est de fluidifier la communication entre la caisse, la cuisine, le service et le client final grâce à une mise à jour en temps réel des statuts de commande.

2. Besoin métier

Dans un restaurant, les erreurs de transmission et le manque de visibilité sur l'état des commandes ralentissent le service.

La solution proposée permet d'enregistrer une commande, de la faire progresser par étapes normalisées et de rendre chaque changement visible immédiatement sur les interfaces concernées.

3. Acteurs et rôles

Rôle	Responsabilité	Interface
Client / Caisse	Créer la commande et simuler le paiement	Application React dédiée
Cuisinier	Passer une commande en préparation puis prête	Application React cuisine
Serveur	Voir les commandes prêtes et marquer servie	Application React service
Client (suivi)	Consulter l'avancement de la commande	Interface de suivi temps réel

4. Fonctionnalités principales

Client / Caisse

- affichage du menu

- ajout au panier
- calcul automatique du total
- validation de la commande
- visualisation du suivi

Cuisinier

- réception instantanée des nouvelles commandes
- passage du statut à en préparation
- passage du statut à prête

Serveur

- vue de toutes les commandes
- mise en avant des commandes prêtes
- passage du statut à servie

Suivi client

- recherche par référence
- frise d'avancement
- rafraîchissement automatique sans rechargement

5. Architecture technique

L'architecture repose sur un backend Node.js + Express qui expose des routes REST pour le menu et les commandes. Socket.io assure la diffusion immédiate des événements métier vers toutes les interfaces connectées. MongoDB est prévu pour la persistance, avec un mode mémoire de secours pour la démonstration locale.

Couche	Choix	Rôle
Front-end	React + Vite	Créer des interfaces distinctes par métier
Back-end	Node.js + Express	Porter la logique métier et les API
Temps réel	Socket.io	Propager les changements de statut
Données	MongoDB + Mongoose	Structurer menu, commandes et utilisateurs

6. Cycle de vie des commandes

Le flux métier partagé entre toutes les interfaces suit quatre statuts simples et compréhensibles :

new → in_preparation → ready → served

Chaque mise à jour déclenche un événement Socket.io afin que l'ensemble des écrans reflète immédiatement l'état courant de la commande.

7. Compétences TP DWWM mobilisées

Bloc de compétences	Illustration dans le projet

Front-end	Maquettage des interfaces, réalisation d’écrans statiques, comportements dynamiques React
Back-end	Conception de modèles, routes API, contrôleurs, composants métier côté serveur
Qualité de code	Séparation des couches, réutilisation de modules partagés, lisibilité et maintenabilité

8. Conclusion

Le projet répond au cahier des charges fonctionnel transmis : il met en scène plusieurs rôles, un suivi temps réel des commandes et une architecture cohérente pour une présentation de fin de parcours. Il peut être complété ensuite par une authentification, une gestion d'administration et un déploiement cloud.